



Namal University Mianwali

Department of Computer Science

Course Code & Title:	CSC-355: Natural Language Processing		
Instructor:	Dr. Muzamil Ahmed	Session / Semester:	2022-2026 8th Semester
Total Marks:	15	Submission Date:	19-03-2026

Student Name: Jalal Ur Rahman

Registration No: NUM-BSCS-2022-42

Assignment 1: NLP Semester Project Proposal

Milestone 1: Project Conceptualization and Preprocessing Design

Section 1: Proposed Project Title

"Sentiment Analysis of Code-Switched Urdu-English Social Media Text Using Multilingual Transformer Models"

The proposed title has been formulated with deliberate precision to communicate the full scope, linguistic challenge, task type, and computational methodology of this research initiative. Each component of the title performs a specific semantic function: the phrase 'Sentiment Analysis' identifies the NLP task (opinion mining and polarity classification); 'Code-Switched Urdu-English' specifies the unique and underexplored linguistic phenomenon being addressed; 'Social Media Text' situates the domain within a high-volume, noisy, real-world data source; and 'Multilingual Transformer Models' signals the advanced computational methodology, namely multilingual pretrained language models such as mBERT and XLM-RoBERTa, that will be employed at the culmination of the project pipeline.

The title is intentionally specific and solution-oriented, avoiding generic labels such as 'Text Analysis System' or 'NLP Classification Project.' It reflects a semester-long technical trajectory: beginning at Milestone 1 with text preprocessing and baseline model design, progressing through word embeddings (FastText, Word2Vec) and sequence models (BiLSTM) in subsequent milestones, and culminating in fine-tuned multilingual Transformer models evaluated with rigorous NLP metrics. This trajectory directly aligns with CLO-3 of the course, which requires students to 'Design NLP solutions using neural networks, Transformers, and pretrained models', ensuring that the project title is not merely a label, but a forward-looking commitment to the full course curriculum.

Section 2: Problem Statement

Pakistani social media platforms including Twitter/X, Facebook, and Instagram are characterised by a pervasive linguistic phenomenon known as code-switching, wherein users seamlessly intersperse Urdu words and phrases within English sentences, and vice versa, within a single post or comment. A representative example is: 'Yeh movie bohat achi thi, definitely watch karo!', which blends Urdu lexicons ('yeh', 'bohat', 'achi', 'karo') with English vocabulary within a single coherent utterance. This produces text that is syntactically ambiguous, phonetically transliterated as Roman Urdu, and linguistically inconsistent at the structural level. Existing NLP sentiment analysis tools, almost universally trained on monolingual English corpora, produce near-random classification results on such mixed-language input, rendering automated opinion mining dangerously unreliable for Pakistani digital markets, brand monitoring systems, and public discourse analysis platforms.

The difficulty of this problem is compounded by several interacting factors that collectively render standard NLP approaches wholly inadequate. Code-switched text has no fixed grammar, no standardised spelling conventions for Roman Urdu (the same Urdu word may appear in dozens of orthographic variants, e.g., 'acha', 'accha', 'acha'), and exhibits extremely high lexical variability. Sentiment polarity can shift mid-sentence between the two language segments, introducing cross-lingual semantic ambiguity. Standard tokenisers, including those in NLTK and spaCy, incorrectly segment transliterated Urdu text; standard English stopword lists miss high-frequency Urdu function words entirely; and word embeddings trained exclusively on English corpora assign zero-vectors or arbitrary representations to Urdu tokens, fundamentally degrading the quality of any downstream model trained on such features.

The necessity of an automated NLP solution cannot be overstated. Pakistan generates millions of social media posts daily, and manual annotation of even a modest sample at scale would be computationally and economically infeasible. Only automated, learned NLP pipelines capable of generalising across unseen, noisy, code-switched inputs can operate at the production scale demanded by commercial and research applications. The computational challenge is therefore substantial: any viable system must simultaneously handle two typologically distinct language families (Indo-Aryan Urdu and Germanic English), resolve transliteration ambiguity, process emoji semantics as sentiment-bearing signals, and learn cross-lingual sentiment representations that no rule-based system could feasibly encode, thereby mandating the use of deep neural language models as the primary computational paradigm.

Section 3: Motivation

Pakistan counts over 70 million active social media users, a number that continues to grow rapidly across urban and rural populations alike. This scale creates an urgent, commercially significant demand for automated sentiment analysis tools capable of processing the unique linguistic reality of Pakistani social media discourse. Businesses require robust brand monitoring systems that can aggregate and classify customer sentiment expressed in code-switched text; political analysts require real-time public opinion tracking on policy decisions articulated in mixed Urdu-English;

academic researchers require labelled sentiment corpora to advance the field of South Asian computational linguistics; and technology startups building Urdu-language AI products require foundational NLP pipelines on which to build downstream applications. All of these use cases are currently blocked by the absence of high-quality, code-switching-aware sentiment analysis tools.

From a research perspective, Urdu-English code-switched sentiment analysis represents a significantly under-explored area in the global NLP literature. The dominant NLP benchmarks including GLUE, SuperGLUE, and SemEval are overwhelmingly English-centric, and most multilingual benchmarks address high-resource language pairs. Building robust, open-source NLP tools for Urdu-English code-switched text would constitute a meaningful contribution to NLP equity, and would establish methodological templates directly applicable to analogous code-switching contexts worldwide including Hindi-English, Arabic-English, Swahili-English, and Mandarin-English mixed language communities. This project therefore carries implications that extend far beyond the immediate Pakistani context. Furthermore, this initiative directly instantiates the course's emphasis on real-world NLP system development, as articulated in Unit 6 of the course outline, which addresses the design, evaluation, and deployment of full-stack NLP systems in authentic, domain-specific settings.

Section 4: Expected Outcomes

The following five measurable and verifiable outcomes are expected upon completion of all five project milestones. Each outcome maps directly to the corresponding milestone in the course schedule (Lectures 10, 16, 22, 27, and 30 respectively):

- **Outcome 1: Text Preprocessing Pipeline (Milestone 1, Due: Lecture 10):** A complete, modular text preprocessing pipeline implemented in Python capable of correctly handling Roman Urdu transliteration, Urdu Nastaliq Unicode script, emojis, Twitter @mentions, hashtags, URLs, bilingual stopwords, and code-switched tokens. The pipeline will be designed to serve as the data preparation layer for all subsequent model experiments across Milestones 2 through 5.
- **Outcome 2: Labelled Sentiment Dataset (Milestone 2, Due: Lecture 16):** A curated and labelled dataset of at least 10,000 code-switched Urdu-English social media posts with sentence-level sentiment annotations (Positive, Negative, Neutral), combining the publicly available SentiMix dataset (SemEval-2020 Task 9) with a self-collected corpus of Pakistani Twitter/X posts gathered via the academic API and manually annotated using an inter-annotator agreement protocol.
- **Outcome 3: Comparative Multi-Model Evaluation (Milestone 3, Due: Lecture 22):** A rigorous comparative evaluation of at least four models spanning the full modelling spectrum covered in the course: (i) Logistic Regression with TF-IDF features (classical baseline); (ii) BiLSTM with FastText multilingual embeddings (neural sequence model); (iii) multilingual BERT (mBERT) with fine-tuning; and (iv) XLM-RoBERTa with fine-

tuning, evaluated using Macro F1-score, Precision, Recall, Accuracy, and Confusion Matrix.

- **Outcome 4: Fine-Tuned Transformer Model (Milestone 4, Due: Lecture 27):** A fine-tuned mBERT or XLM-RoBERTa model achieving a minimum target Macro F1-score of 0.78 on the held-out test set, with documented hyperparameter tuning, learning rate scheduling, and class-imbalance mitigation strategies applied during the fine-tuning process.
- **Outcome 5: Final Evaluation and Deployment Report (Milestone 5, Due: Lecture 30):** A comprehensive written evaluation report documenting model performance comparisons across all four architectures; an error analysis of mispredicted samples; a bias and fairness assessment; an estimation of computational resource requirements; and a feasibility analysis for deploying the fine-tuned model as a lightweight inference API or mobile on-device service.

Section 5: Dataset Identification

5.1 Primary Dataset: SentiMix (SemEval-2020 Task 9)

The primary dataset for this project is SentiMix, the official corpus released for SemEval-2020 Task 9: Sentiment Analysis for Code-Mixed Social Media Text. This is a publicly available, peer-reviewed benchmark dataset comprising approximately 15,000 labelled tweets in English-Hindi code-switched text, the most linguistically proximate publicly available resource to Urdu-English code-switching, given that Hindi and Urdu share the same grammatical structure and substantial lexical overlap (both are derived from Khari Boli). Sentence-level sentiment annotations are provided, with three classes: Positive, Negative, and Neutral. The dataset is available at: <https://competitions.codalab.org/competitions/20911>. The use of a competitive benchmark dataset ensures reproducibility and enables direct comparison with published results in the SemEval proceedings.

5.2 Secondary Dataset: Self-Collected Pakistani Twitter/X Corpus

To supplement SentiMix with authentic Urdu-English code-switched content specific to Pakistani social media discourse, a secondary corpus of approximately 3,000-5,000 posts will be collected from Twitter/X using the Academic Research Product Track API, applying keyword filters for high-frequency Urdu-English mixed terms across domains including film reviews, political commentary, product feedback, and sports. Manual sentiment annotation will be performed using a standardised three-label scheme (Positive, Negative, Neutral), with inter-annotator agreement measured via Cohen's Kappa statistic. Language data includes both Roman Urdu script (transliterated) and Urdu Nastaliq Unicode script (U+0600-U+06FF). No token-level language identification labels are provided in the raw corpora; language identification per token will be performed as an automated preprocessing step using Unicode character range heuristics.

5.3 Dataset Quality and Generalization

Both dataset size and annotation quality directly govern the generalisation capacity of any supervised learning model trained on them, as discussed extensively in Yoav Goldberg's Neural Network Methods for Natural Language Processing. Insufficient or inconsistently labelled data produces models that memorise training artifacts rather than learning transferable sentiment representations. This risk will be mitigated in this project through careful annotation protocols, dataset stratification, and the application of cross-lingual transfer learning via pretrained multilingual models that have been exposed to large-scale multilingual corpora during pretraining.

Section 6: Required NLP Text Preprocessing

The preprocessing pipeline for code-switched Urdu-English social media text is substantially more complex than standard monolingual English pipelines. The following ten steps are required, each justified specifically with respect to the linguistic properties of the target data. Note that in Milestone 3, the custom tokenisation stage will be partially superseded by mBERT's built-in WordPiece subword tokeniser; however, the full pipeline below is essential for baseline models in Milestones 1 and 2.

Step 1: Unicode Normalization. Urdu text in digital form exists across multiple, visually identical but byte-level distinct Unicode representations, specifically NFC (Canonical Decomposition followed by Canonical Composition) and NFD (Canonical Decomposition). Without Unicode normalisation to a single canonical form, the identical Urdu word may be represented as two distinct byte sequences, causing it to be treated as two separate vocabulary items by any downstream tokeniser or embedding lookup. Unicode normalisation to NFC is therefore a non-negotiable prerequisite for any Urdu script processing pipeline.

Step 2: Lowercasing (English Tokens Only). Uppercase variation in English words ('Happy', 'HAPPY', 'happy') introduces artificial vocabulary expansion without semantic benefit. Lowercasing normalises these variants to a single type, reducing vocabulary size and improving embedding quality. Crucially, this operation affects only Latin-script (English and Roman Urdu) characters; Urdu Nastaliq script is inherently case-insensitive and is unaffected by this transformation.

Step 3: URL Removal. Social media posts frequently contain hyperlinks that carry zero intrinsic sentiment information but introduce highly variable token sequences that confuse both statistical and neural models. All URLs matching standard HTTP/HTTPS patterns are removed using regular expression filters before any tokenisation is performed.

Step 4: Mention and Hashtag Cleaning. @username mentions are entirely uninformative for sentiment classification and must be removed. Hashtag symbols (#) are stripped, but the hashtag word itself is retained, as the content frequently carries sentiment-bearing meaning (e.g., '#Disappointed' becomes 'disappointed'). This nuanced handling is specific to social media NLP pipelines and has no equivalent in standard document corpora.

Step 5: Emoji Handling. Emojis are among the strongest sentiment signals in informal social media text. A naive approach of simply deleting emojis would discard this rich sentiment information entirely. Instead, emojis are converted to descriptive text labels using the Python emoji library's `demojize()` function (e.g., a smiling face emoji is converted to 'smiling_face', a crying face to 'crying_face'), preserving their semantic content in a form that downstream models can process as standard tokens.

Step 6: HTML Entity and Noise Removal. Web-scraped and API-collected social media text frequently contains HTML entities, residual XML tags, and non-linguistic special characters. These must be removed using targeted regular expression patterns. Critically, the removal pattern must preserve the Urdu Unicode character range (U+0600-U+06FF) and sentiment-bearing punctuation marks (! and ?) to avoid inadvertent information loss.

Step 7: Custom Tokenisation. Standard NLP tokenisers, including both NLTK's `word_tokenize` and spaCy's default tokeniser, are trained exclusively on English text and produce systematically incorrect segmentation for Urdu tokens and code-switched sequences. A hybrid tokenisation approach based on whitespace splitting supplemented by regex pattern matching is therefore employed for the baseline model stages. In later milestones (from Milestone 3 onwards), mBERT's multilingual WordPiece tokeniser, which has been trained on 104 languages including both Urdu and English, will provide a far superior subword-level segmentation that naturally handles out-of-vocabulary Urdu tokens.

Step 8: Bilingual Stopword Removal. Standard English stopword lists, as provided by NLTK, contain no Urdu function words. In code-switched text, high-frequency Urdu function words contribute no discriminative sentiment signal yet would otherwise consume valuable vocabulary capacity. A custom bilingual stopword list, combining NLTK's English stopwords with a curated Urdu stopword lexicon, is required to adequately reduce noise across both language components.

Step 9: Punctuation and Number Filtering. Pure numeric tokens (stand-alone digit strings) carry no sentiment value in this context and are removed. Standard punctuation is filtered, but sentiment-bearing punctuation characters, most significantly the exclamation mark (!) and question mark (?), are retained as potentially discriminative features, given their frequent association with strong emotional expression in informal social media writing.

Step 10: Language Identification per Token. Identifying the language of each individual token, either Urdu (UR) or English (EN), is a preprocessing step that is entirely unique to code-switched NLP pipelines and has no direct analogue in monolingual processing. A Unicode character-range heuristic is applied: tokens containing characters within U+0600-U+06FF are classified as Urdu; tokens containing predominantly ASCII characters are classified as English. This per-token language tag enables language-conditional feature extraction and will serve as the basis for language-aware analysis in subsequent milestones.

Section 7: Python Preprocessing Code

The following Python implementation provides a complete, modular, and fully commented preprocessing pipeline for code-switched Urdu-English social media text. All functions correspond directly to the preprocessing steps described in Section 6. Sample inputs and outputs from diverse text types are included in the demonstration block.

7.1 Full Implementation

```
# =====
# NLP Preprocessing Pipeline
# Project: Sentiment Analysis of Code-Switched Urdu-English Text
# Course: CSC-355 Natural Language Processing
# Namal University Mianwali
# =====

import re
import string
import unicodedata
import emoji
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import nltk

nltk.download('stopwords', quiet=True)
nltk.download('punkt', quiet=True)

# Custom Urdu Stopword List
URDU_STOPWORDS = {
    'بے', 'بیں', 'کا', 'کی', 'کے', 'میں', 'نے', 'سے', 'پر',
    'اور', 'یہ', 'وہ', 'ایک', 'بھی', 'تو', 'کم', 'جو', 'بو',
    'اس', 'اب', 'تھا', 'تھی', 'تھے', 'ہم', 'آپ', 'مجھے',
    'he', 'hein', 'ka', 'ki', 'ke', 'mein', 'ne', 'se', 'par',
    'aur', 'yeh', 'woh', 'ek', 'bhi', 'to', 'jo', 'ho',
    'is', 'ab', 'tha', 'thi', 'the', 'hum', 'aap', 'mujhe'
}

ENGLISH_STOPWORDS = set(stopwords.words('english'))

# Step 1: Unicode Normalization
def normalize_unicode(text: str) -> str:
    # Normalize Unicode to NFC -- critical for consistent Urdu handling.
    return unicodedata.normalize('NFC', text)

# Step 2: Emoji Handling
def convert_emojis(text: str) -> str:
    # Convert emojis to descriptive text labels (e.g., smiling_face).
    # Emojis are strong sentiment signals -- removing them loses information.
    return emoji.demojize(text, delimiters=(" ", " "))

# Step 3: URL Removal
def remove_urls(text: str) -> str:
    # Remove all URLs -- they carry no sentiment value.
    return re.sub(r'http\S+|www.\S+', '', text)

# Step 4: Mention and Hashtag Cleaning
def clean_mentions_hashtags(text: str) -> str:
    # Remove @mentions entirely. Strip # from hashtags but keep the word.
```

```

    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'#(\w+)', r'\1', text)
    return text

# Step 5: Lowercasing (English only)
def lowercase_english(text: str) -> str:
    # Lowercase the text -- affects only English/Roman characters.
    return text.lower()

# Step 6: Noise Removal
def remove_noise(text: str) -> str:
    # Remove HTML entities and special characters.
    # Preserve Urdu Unicode range (U+0600-U+06FF) and sentiment punct (! ?)
    text = re.sub(r'<.*?>', '', text)
    text = re.sub(r'^\w\s\u0600-\u06FF!?', ' ', text)
    return text

# Step 7: Tokenization
def tokenize(text: str) -> list:
    # Tokenize using NLTK word_tokenize.
    # Note: mBERT's WordPiece tokenizer will replace this in Milestone 3.
    return word_tokenize(text)

# Step 8: Bilingual Stopword Removal
def remove_stopwords(tokens: list) -> list:
    # Remove both English and Urdu stopwords.
    all_stopwords = ENGLISH_STOPWORDS.union(URDU_STOPWORDS)
    return [token for token in tokens if token not in all_stopwords]

# Step 9: Number and Punctuation Filtering
def filter_numbers_and_punct(tokens: list) -> list:
    # Remove pure number tokens. Retain ! and ? for sentiment value.
    return [t for t in tokens if not t.isdigit() and t.strip()]

# Step 10: Language Identification per Token
def tag_language(tokens: list) -> list:
    # Tag each token as 'UR' (Urdu) or 'EN' (English) via Unicode heuristic.
    tagged = []
    for token in tokens:
        if any('\u0600' <= ch <= '\u06FF' for ch in token):
            tagged.append((token, 'UR'))
        else:
            tagged.append((token, 'EN'))
    return tagged

# Full Pipeline
def preprocess(text: str, verbose: bool = True) -> list:
    # Complete preprocessing pipeline for code-switched Urdu-English text.
    if verbose:
        print(f"\n{'='*55}")
        print(f"INPUT : {text}")
    text = normalize_unicode(text)
    text = convert_emojis(text)
    text = remove_urls(text)
    text = clean_mentions_hashtags(text)
    text = lowercase_english(text)
    text = remove_noise(text)
    tokens = tokenize(text)
    tokens = remove_stopwords(tokens)
    tokens = filter_numbers_and_punct(tokens)
    tagged_tokens = tag_language(tokens)

```

```

    if verbose:
        print(f"OUTPUT: {tagged_tokens}")
    return tagged_tokens

# Sample Input / Output Demonstration
sample_texts = [
    "Yeh film bohat achi thi! Must watch karo #Lollywood @user123",
    "https://t.co/abc",
    "گورنمنٹ نے کچھ نہیں کیا اس مسئلے پر",
    "Absolutely LOVED the new drama! Urdu dramas are unmatched #Drama2024"
]

print("PREPROCESSING PIPELINE - DEMONSTRATION")
for text in sample_texts:
    result = preprocess(text)

```

7.2 Pipeline Step Summary Table

The table below summarises each pipeline function, its purpose, and the specific reason it is necessary for code-switched Urdu-English NLP, as distinct from generic monolingual pipelines.

Step	Function	General Purpose	Code-Switch-Specific Necessity
1	normalize_unicode()	Consistent character encoding	Multiple Unicode representations of identical Urdu characters cause spurious vocabulary duplication
2	convert_emojis()	Preserve sentiment from emojis	Emojis are primary sentiment signals in informal Urdu-English social media text
3	remove_urls()	Eliminate noise tokens	URLs appear at high frequency in tweets; produce unique tokens with zero sentiment value
4	clean_mentions_hashtags()	Remove/normalise social media markers	Hashtag content often carries sentiment; stripping only the # symbol retains it
5	lowercase_english()	Normalise English case	Affects only Latin characters; Urdu Nastaliq is inherently case-insensitive
6	remove_noise()	Strip HTML and special characters	Must explicitly preserve Urdu Unicode range (U+0600-U+06FF) to avoid losing Urdu tokens
7	tokenize()	Split text into tokens	Standard tokenisers fail on Urdu; mBERT WordPiece replaces this in Milestone 3
8	remove_stopwords()	Reduce vocabulary noise	Requires bilingual stopword list — English-only lists miss all Urdu function words
9	filter_numbers_and_punct()	Remove non-informative tokens	Sentiment punctuation (! ?) retained as discriminative features for social media text
10	tag_language()	Identify language per token	Unique to code-switched NLP — enables language-conditional analysis in later milestones

Section 8: Why This Is a Complex Computing Problem (CCP Analysis)

According to the assignment guidelines, a qualifying Complex Computing Problem must satisfy a defined set of attributes relating to data scale, computational non-triviality, ambiguity, algorithmic sophistication, evaluation rigour, real-world applicability, and multi-component system integration. The seven CCP attributes are addressed below with specific reference to this project:

- **1. Large Scale Data Handling:** The primary corpus contains over 15,000 publicly sourced tweets supplemented by a self-collected dataset of 3,000-5,000 posts, requiring multi-stage preprocessing across heterogeneous data sources in two scripts and two language families.
- **2. Non Trivial Computation:** The project pipeline spans five progressively complex processing stages across the full semester, from rule-based preprocessing through classical machine learning, to neural sequence models (BiLSTM), to fine-tuning large pretrained Transformer models, none of which can be trivially implemented or substituted with off-the-shelf tools.
- **3. Ambiguity Handling:** The system must simultaneously resolve code-switching boundaries, Roman Urdu orthographic ambiguity (multiple phonetic spellings per word), emoji polysemy, and cross-lingual sentiment polarity shifts, representing multiple interacting layers of linguistic ambiguity that demand learned, probabilistic representations rather than deterministic rule systems.
- **4. Algorithmic Design:** The project requires the selection, design, and comparison of four distinct model architectures: TF-IDF with Logistic Regression, BiLSTM with FastText embeddings, fine-tuned mBERT, and fine-tuned XLM-RoBERTa, each embodying fundamentally different algorithmic assumptions about language representation and generalisation.
- **5. Performance Evaluation:** Model performance will be assessed using a comprehensive suite of standard NLP metrics: Macro F1-score (primary metric for class-imbalanced data), Precision, Recall, Accuracy, and Confusion Matrix analysis, consistent with the evaluation framework presented in the Handbook of Natural Language Processing and covered in Unit 6 of the course outline.
- **6. Real World Constraints:** This project directly addresses an active, commercially relevant, and unresolved problem in Pakistani social media analytics, operating under real data constraints including annotation cost, API rate limits, class imbalance, and the need for deployment-viable model sizes.
- **7. Multiple Components Integration:** The final system integrates six distinct functional components: data collection and curation, custom multilingual preprocessing, classical feature engineering, neural model training with transfer learning, rigorous cross-architecture evaluation, and deployment feasibility analysis, constituting a full, end-to-end NLP system of substantial engineering and research complexity.

Section 9: References

1. Lee, R. (2022). Natural Language Processing (1st ed.).
2. Goldberg, Y. (2017). Neural Network Methods for Natural Language Processing (1st ed.). Morgan & Claypool Publishers.
3. Kamath, U., Graham, K., & Chiu, W. (2024). Large Language Models: A Deep Dive. Springer.
4. Dale, R., Moisl, H., & Somers, H. (Eds.). (2000). Handbook of Natural Language Processing (2nd ed.). Chapman & Hall/CRC.
5. Patwa, P., Aguilar, G., Kar, S., et al. (2020). SentiMix 2020: Sentiment Analysis for Code-Mixed Social Media Text. Proceedings of the Fifth Workshop on Computational Approaches to Linguistic Code-Switching, EMNLP 2020.
6. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT 2019.
7. Conneau, A., et al. (2020). Unsupervised Cross-lingual Representation Learning at Scale. Proceedings of ACL 2020.